🐍 الدليل الشامل لأساسيات البرمجة بلغة بايثون (Python)

> فكرة هذا الملف كُتب هذا الدليل بلغة مبسطة ومباشرة ليكون مرجعاً شاملاً لأي طالب، حتى لو لم يكتب سطراً برمجياً واحداً في حياته. يركز الملف على **فهم الفكرة المنطقية** خلف كل أمر برمجي، ثم **طريقة تحويل السؤال الرياضي أو المنطقي إلى كود خطوة بخطوة**.

أولاً: أمر الطباعة `print` وأمر الإدخال `input`

يُستخدم الأمر `print()` لإظهار النتائج أو النصوص على الشاشة للمستخدم. وتختلف طريقة عمله بحسب ما نضعه بين قوسيه:

الحالة الأولى: طباعة نص حرفي كما هو (String)

يُستخدم لطباعة أي نص أو جملة بشكل حرفي تماماً كما كُتبت، بشرط وضع النص بين علامتي اقتباس مزدوجة `" "` أو مفردة `' '`.

```python
print("Hello")
```

- **الناتج على الشاشة:** `Hello`
- **التفسير:** طالما أن الكلمة محصورة بين علامتي اقتباس `" "`، فإن بايثون لا تحاول فهم معناها أو البحث عن قيمتها، بل تطبعها كحروف كما هي.

الحالة الثانية: طباعة قيمة متغير (Variable)

المتغير هو بمثابة "صندوق" في الذاكرة نسميه باسم معين (مثل `x`) ونخزن بداخله قيمة (مثل `5`). لطباعة القيمة الموجودة داخل المتغير، نكتب اسم المتغير داخل `print()` **بدون علامات اقتباس**.

```python
x = 5
print(x)
```

- **الناتج على الشاشة:** `5`

> ⚠ **انتبه للفرق الجوهري بين الحالتين!**
>
> - إذا كتبنا `print(x)` بدون علامات اقتباس ← يبحث البرنامج عن قيمة `x` في الذاكرة ويطبع `5`.
> - إذا كتبنا `print("x")` مع علامات اقتباس ← يتعامل معها كنص حرفي ويطبع الحرف `x`.

الحالة الثالثة: طباعة أكثر من جزء في نفس الأمر (دمج النصوص والمتغيرات)

إذا أردنا طباعة نص توضيحي وبجانبه قيمة متغير في سطر واحد، نفصل بين كل جزء وآخر بفاصلة عادية `,`. تقوم الفاصلة بطباعة كل جزء بالترتيب مع ترك مسافة صغيرة تلقائياً بينهما.

```python
x = 5
print("x =", x)
```

- **الناتج على الشاشة:** `x = 5`
- **التفسير:**
  1. الجزء الأول `"x ="` موضوع بين علامتي اقتباس، لذلك طُبع حرفياً كما هو.
  2. الجزء الثاني `x` بدون علامات اقتباس، لذلك طبع البرنامج قيمته المخزنة وهي `5`.

الحالة الرابعة: إجراء العمليات الحسابية مباشرة داخل `print`

يمكن لأمر `print()` أن يعمل كآلة حاسبة؛ فإذا وضعنا عملية رياضية بداخله **بدون علامات اقتباس**، يقوم بحساب الناتج أولاً ثم يطبعه مباشرة.

PYTHON

```python
print(5 + 7)
print(10 - 3)
print(4 * 2)
print(8 / 2)
```

- **النواتج بالترتيب:** `12` ثم `7` ثم `8` ثم `4.0`

جدول العمليات الحسابية في بايثون (مهم جداً في المسائل):

العملية	الرمز في بايثون	مثال	الناتج	ملاحظة توضيحية
الجمع	`+`	`5 + 7`	`12`	جمع اعتيادي
الطرح	`-`	`10 - 4`	`6`	طرح اعتيادي
الضرب	`*`	`3 * 4`	`12`	**انتبه:** لا يجوز كتابة `3x` بل يجب كتابة `3 * x`
القسمة العادية	`/`	`7 / 2`	`3.5`	تعطي الناتج الكامل مع الفاصلة العشرية
القسمة الصحيحة	`//`	`7 // 2`	`3`	تحذف الكسر وتعطي العدد الصحيح فقط
باقي القسمة	`%`	`7 % 2`	`1`	تعطي الباقي من القسمة (مفيدة جداً لمعرفة الزوجي والفردي)
الأس (القوة)	`**`	`3 ** 2`	`9`	تعني 3^2 (أي 3 أس 2)

تمهيد ضروري للمسائل: كيف ندخل قيماً للبرنامج باستخدام `input()` ؟

في المسائل البرمجية، لا نكتب دائماً `x = 5` بشكل ثابت، بل نطلب من المستخدم أن يدخل الرقم بنفسه من لوحة المفاتيح أثناء تشغيل البرنامج. هنا نستخدم دالة `input()` :

1. **لإدخال عدد صحيح (Integer):** نستخدم `int(input("رسالة توضيحية"))` .
2. **لإدخال عدد عشري (Float):** نستخدم `float(input("رسالة توضيحية"))` .
3. **لإدخال نص (String):** نستخدم `input("رسالة توضيحية")` مباشرة بدون `int` .

> 💡 **لماذا نكتب `int( ... )` قبل `input` ؟ لأن دالة `input()` وحدها تقرأ أي شيء يكتبه المستخدم كنص (`"5"` وليس الرقم `5`). ولكي نتمكن من جمع الرقم أو ضربه في المعادلات، نضع `int()` لتحويله إلى عدد صحيح قابل للحساب.**

تطبيق شامل على المعادلات والإدخال والطباعة

مثال 1 (من المحاضرة): إيجاد قيمة معادلة معتمدة على معادلة أخرى

السؤال:

```
W.P. (Write a Program) to find the value of y: y = 3B + 2x, B = 4M - 2
```

طريقة التفكير وتفكيك السؤال قبل كتابة الكود:

1. المطلوب النهائي في السؤال هو حساب وطباعة قيمة المتغير `y` من المعادلة: $y = 3B + 2x$.
2. لكي نحسب `y` ، نحتاج إلى معرفة قيمة `B` وقيمة `x` .
3. نلاحظ أن السؤال أعطانا معادلة جاهزة لحساب `B` وهي: $B = 4M - 2$.
4. إذن، المجاهيل الأساسية التي لا نملك لها معادلة ونحتاج إدخالها من لوحة المفاتيح هي `x` **و** `m` **فقط**.

خطوات الحل بالترتيب:

- **الخطوة الأولى:** نطلب من المستخدم إدخال قيمتي `x` و `m` باستخدام `int(input( ... ))` .
- **الخطوة الثانية:** نحسب قيمة `b` من معادلتها (مع الانتباه لكتابة علامة الضرب `*` بين `4` و `m`).
- **الخطوة الثالثة:** نحسب قيمة `y` من معادلتها بعد أن أصبحت `b` و `x` معلومتين لدينا.
- **الخطوة الرابعة:** نطبع الناتج النهائي لـ `y` .

PYTHON

```python
# الخطوة 1: إدخال قيم المجاهيل الأساسية
x = int(input("Give the value of x: "))
m = int(input("Give the value of m: "))

# الخطوة 2: استخراج قيمة b من المعادلة المعطاة
b = 4 * m - 2

# الخطوة 3: استخراج قيمة y من المعادلة الرئيسية
y = 3 * b + 2 * x

# الخطوة 4: طباعة النتيجة النهائية
print("y =", y)
```

تجربة الكود بالأرقام (تتبع الحل): لو أدخل الطالب `x = 2` و `m = 3` :

1. يحسب البرنامج `b = 4 * 3 - 2` ← تصبح قيمة `b` تساوي `10` .
2. يحسب البرنامج `y = 3 * 10 + 2 * 2` ← تصبح قيمة `y` تساوي `34` .
3. يطبع على الشاشة: `y = 34` .

مثال 2 (إضافي): معادلة تحتوي على أسس وقسمة

السؤال: `W.P. to find the value of Z where: Z = (x^2 + 5y) / 2`

التفكير بالحل:

- المجهولان هما `x` و `y` ، ندخلهما أولاً.
- نضع البسط كاملًا بين قوسين `(x ** 2 + 5 * y)` لأن القسمة تشمل المقدار كله، ثم نقسم على `2` .

PYTHON

```python
x = float(input("Enter x: "))
y = float(input("Enter y: "))

z = (x ** 2 + 5 * y) / 2

print("Z =", z)
```

ثانياً: الجمل الشرطية `if / elif / else`

تُستخدم الجملة الشرطية عندما نريد من البرنامج أن يتخذ قراراً وينفذ كوداً معيناً **فقط إذا تحقق شرط محدد**.

القاعدة العامة لجملة `if` :

تتكون جملة `if` من: **(أداة الشرط `if` + الشرط المنطقي + نقطتين رأسيتين `:` + جواب الشرط مع مسافة بادئة)**.

```python
if الشرط:
    جواب الشرط (الأمر الذي يتنفذ إذا كان الشرط صحيحاً)
```

> **قاعدتان ذهبيتان في جملة `if` :**
>
> 1. **النقطتان الرأسيتان** `:` في نهاية سطر `if` و `elif` و `else` إجبارية، ونسيانها يسبب خطأ برمجياً.
> 2. **المسافة البادئة (** `Indentation` **/ زر** `Tab` **):** الأسطر التابعة لجواب الشرط يجب أن تدخل قليلاً إلى اليمين (بمقدار 4 مسافات أو ضغطة `Tab`). هكذا تعرف بايثون أن هذا السطر تابع لجملة `if` .

رموز المقارنة في الشروط:

الرمز	المعنى الرياضي	مثال	متى يكون الشرط صحيحاً؟
`>`	أكبر من	`x > 0`	إذا كانت `x` أكبر من الصفر
`<`	أصغر من	`x < 6`	إذا كانت `x` أصغر من 6
`≥`	أكبر من أو يساوي	`x ≥ 50`	إذا كانت `x` تساوي 50 أو أكثر
`≤`	أصغر من أو يساوي	`x ≤ 10`	إذا كانت `x` تساوي 10 أو أقل
`==`	**هل يساوي؟ (مقارنة)**	`x == 0`	إذا كانت قيمة `x` تساوي صفر
`≠`	لا يساوي	`x ≠ 5`	إذا كانت قيمة `x` أي رقم غير 5

الحالة الأولى: جملة `if` مع `else` (شرط واحد واحتمالان)

نستخدم `if` لفحص الشرط، ونستخدم `else` (ومعناها: **وإلا**) لتحديد ما يفعله البرنامج إذا لم يتحقق الشرط الأول.

```python
x = 4

if x > 0:
    print(x)
else:
    print("H")
```

- **شرح الكود خطوة بخطوة:**
  1. أعطينا المتغير `x` القيمة `4` .
  2. `if x > 0:` معناه: افحص قيمة `x` ، هل هي أكبر من `0` ؟ فإذا تحقق الشرط اطبع قيمة `x` .
  3. `else:` معناه: إذا لم يتحقق الشرط الأول (أي كانت `x` صفراً أو سالبة)، انتقل هنا واطبع الحرف `"H"` .
  4. **النتيجة هنا:** بما أن `4` أكبر من `0` ، يتحقق الشرط الأول فيطبع `4` ويتجاهل `else` تماماً.

الحالة الثانية: تعدد الشروط باستخدام `elif`

إذا كان لدينا أكثر من شرطين، نستخدم `elif` (وهي اختصار لعبارة `else if` أي: **وإلا إذا**).

```python
x = 4

if x > 3:
    print("1")
elif x < 6:
    print("2")
else:
    print("3")
```

- **شرح الكود وطريقة عمل بايثون:**
  1. يبدأ البرنامج بفحص الشرط الأول `if x > 3:` .
  2. إذا لم يتحقق الأول، ينتقل لفحص الشرط الثاني `elif x < 6:` .
  3. إذا لم يتحقق أي شرط مما سبق، ينتقل تلقائياً إلى `else:` الأخيره وينفذها.

> ملاحظة ذكية جداً على المثال أعلاه (انتبه لها في الامتحان!): عندما تكون `x = 4` ، نلاحظ أن `4` أكبر من `3` (الشرط الأول صحيح) وأيضاً `4` أصغر من `6` (الشرط الثاني صحيح)! **فماذا سيطبع البرنامج؟** سيطبع `1` **فقط** ويتوقف! لأن سلسلة `if ... elif ... else` مرتبطة ببعضها؛ بمجرد أن يجد البرنامج **أول شرط صحيح** ينفذه ويخرج من السلسلة بالكامل دون النظر إلى الشروط التالية.

الحالة الثالثة: الشروط المتداخلة (`Nested if`)

ومعناها وضع جملة `if` كاملة **داخل** جملة `if` أخرى. نستخدمها عندما نريد فحص شرط أول، وبناءً على نتيجته نفحص شرطاً ثانياً بالداخل.

مثال (من المحاضرة): إيجاد العدد الأكبر بين 3 أعداد `x, y, z` باستخدام `Nested if`

السؤال:

```
W.P. to find the largest number among three numbers (x, y, z) using Nested if.
```

منطق الحل قبل الكود: لدينا ثلاثة أعداد `x` و `y` و `z` ونريد معرفة من هو الأكبر بينها:

1. نقارن أولاً بين `x` و `y` عبر الشرط الخارجي `if x > y:` :

- **إذا كان `x > y` صحيحاً:** هذا يعني أن `y` خرج من المنافسة لأنه أصغر من `x` . تبقى المنافسة محصورة بين `x` و `z` ! لذلك نضع بالداخل `if x > z:` فإذا تحقق يكون `x` هو الأكبر، وإلا (`else`) يكون `z` هو الأكبر.
- **إما إذا كان `x > y` غير صحيح (`else` الخارجية):** هذا يعني أن `x` خرج من المنافسة لأن `y` أكبر منه. تبقى المنافسة محصورة بين `y` و `z` ! فنضع بداخلها `if y > z:` فإذا تحقق يكون `y` هو الأكبر، وإلا (`else`) يكون `z` هو الأكبر.

PYTHON

```python
x = int(input("Give the value of x: "))
y = int(input("Give the value of y: "))
z = int(input("Give the value of z: "))

if x > y:
    # إذا وصلنا هنا، فنحن متأكدون أن x أكبر من y، فنقارن x مع z
    if x > z:
        print(x)
    else:
        print(z)
else:
    # إذا وصلنا هنا، فهذا يعني أن y (أو يساوي) أكبر من x، فنقارن y مع z
    if y > z:
        print(y)
    else:
        print(z)
```

الحالة الرابعة (إضافية): دمج الشروط في سطر واحد باستخدام `and` و `or`

بدلاً من تكرار `if` داخل `if`، يمكننا في كثير من المسائل ربط شرطين معاً في سطر واحد:

- `and` **(وَ):** يشترط أن يكون الشرطان صحيحين معاً في نفس الوقت.
- `or` **(أو):** يكفي أن يكون أحد الشرطين صحيحاً على الأقل.

مثال إضافي 1: حل نفس سؤال أكبر عدد بين 3 أعداد باستخدام `and` و `elif`

PYTHON

```python
x = int(input("Give the value of x: "))
y = int(input("Give the value of y: "))
z = int(input("Give the value of z: "))

if x >= y and x >= z:
    print("The largest is:", x)
elif y >= x and y >= z:
    print("The largest is:", y)
else:
    print("The largest is:", z)
```

مثال إضافي 2 (مهم جداً للامتحانات): فحص هل العدد زوجي أم فردي (Even or Odd)

السؤال: `W.P. to check if a number is Even or Odd.`

الفكرة: العدد الزوجي هو كل عدد يقبل القسمة على `2` بدون باقٍ (أي أن باقي قسمته على 2 يساوي صفراً: `x % 2 == 0`).

PYTHON

```python
x = int(input("Enter a number: "))

if x % 2 == 0:
    print("Even (عدد زوجي)")
else:
    print("Odd (عدد فردي)")
```

مثال إضافي 3: فحص هل العدد موجب، أم سالب، أم صفر

السؤال: `W.P. to check whether a number is positive, negative, or zero.`

PYTHON

```python
num = int(input("Enter a number: "))

if num > 0:
    print("Positive (موجب)")
elif num < 0:
    print("Negative (سالب)")
else:
    print("Zero (صفر)")
```

ثالثاً: حلقات التكرار `for` (`for Loop`)

تُستخدم حلقة `for` لتكرار تنفيذ أمر أو مجموعة أوامر لعدد محدد من المرات، بدلاً من إعادة كتابة نفس الكود يدوياً عشرات المرات.

القاعدة العامة لحلقة `for` :

```python
for متغير_العداد in range(إعدادات_النطاق):
    الأوامر المراد تكرارها
```

الحالات الثلاث لدالة `range()` :

دالة `range()` هي المسؤولة عن توليد سلسلة الأرقام التي سيمشي عليها العداد `i` . ولها 3 أشكال:

الحالة	الصيغة	من أين تبدأ؟	متى تتوقف؟	مقدار التغير (الخطوة)	مثال	الأرقام التي يأخذها `i`
1. النهاية فقط	`range( النهاية )`	من `0` تلقائياً	**قبل النهاية بـ 1**	زيادة `+1` تلقائياً	`range(5)`	`0, 1, 2, 3, 4` (5 دورات)
2. البداية والنهاية	`range( البداية , النهاية )`	من `البداية`	**قبل النهاية بـ 1**	زيادة `+1` تلقائياً	`range(1, 6)`	`1, 2, 3, 4, 5`
3. البداية والنهاية والتغير	`range( البداية , النهاية , التغير )`	من `البداية`	**قبل النهاية**	حسب رقم `التغير`	`range(2, 11, 2)`	`2, 4, 6, 8, 10`

⚠ **القاعدة الأهم في `range` : رقم النهاية لا يدخل أبداً! دائماً تتوقف `range` قبل رقم النهاية بواحد:**

- إذا أردت الأعداد من `1` إلى `10` ← تكتب `range(1, 11)` .
- إذا أردت تكرار كود `n` من المرات ← تكتب `range(n)` (فيبدأ العداد من `0` إلى `n - 1` ويكون مجموع الدورات `n` بالضبط).
- للعد التنازلي (من الكبير للصغير) نضع مقدار التغير بالسالب، مثل: `range(10, 0, -1)` فيعطينا `10, 9, 8, ... , 1` .

أمثلة وتطبيقات شاملة على `for`

المثال الأول: إيجاد مجموع الأعداد من 1 إلى 10

السؤال: `Ex 1: W.P. to find the sum of numbers from 1 to 10`

طريقة التفكير بالحل:

1. نحتاج إلى وعاء (أو حصّالة) نجمع فيها الأرقام واحداً تلو الآخر. نسمي هذا المتغير `s` ونضع قيمته الابتدائية `s = 0` **قبل بدء الحلقة** (لأن الصفر هو العنصر المحايد للجمع فلا يغير النتيجة).
2. ننشئ حلقة `for` تبدأ من `1` وتنتهي عند `11` لكي تعطينا الأرقام من `1` إلى `10` .
3. في كل دورة، نضيف الرقم الحالي `i` إلى الحصّالة `s` باستخدام الأمر `s += i` (وهي اختصار لـ `s = s + i`).
4. بعد انتهاء الحلقة تماماً، نطبع المجموع النهائي `s` .

PYTHON

```python
s = 0

for i in range(1, 11):
    s += i    # يكافئ تماماً كتابة: s = s + i

print("s =", s)
```

- **الناتج النهائي:** `s = 55`

كيف دارت الحلقة في الذاكرة؟ (جدول تتبع لـ 4 دورات أولى):

- قبل الحلقة: `s = 0`
- الدورة 1 (`i = 1`): `s = 0 + 1` ← تصبح `s = 1`
- الدورة 2 (`i = 2`): `s = 1 + 2` ← تصبح `s = 3`
- الدورة 3 (`i = 3`): `s = 3 + 3` ← تصبح `s = 6`
- الدورة 4 (`i = 4`): `s = 6 + 4` ← تصبح `s = 10`
- ... وهكذا حتى الدورة 10 (`i = 10`): `s = 45 + 10` ← تصبح `s = 55`.

المثال الثاني: إيجاد المعدل (المتوسط الحسابي) لعشرة أعداد

السؤال: `Ex 2: W.P. to find the average of ten numbers`

القانون الرياضي: المعدل (`Average`) = **مجموع الأعداد ÷ عددها**. في الامتحانات قد يُقصد بهذا السؤال إحدى حالتين، وسنشرحهما معاً:

الحالة (أ): معدل 10 أعداد يدخلها المستخدم بنفسه (الأكثر شيوعاً في الامتحانات)

الخطوات:

1. نصفر متغير المجموع `s = 0`.
2. نكرر الحلقة `10` مرات باستخدام `range(10)`.
3. في كل دورة نطلب من المستخدم إدخال رقم `x`، ثم نضيف `x` فوراً إلى المجموع `s`.
4. بعد انتهاء الـ 10 دورات، نقسم المجموع `s` على `10` لنحصل على المعدل `avg`.

PYTHON

```python
s = 0

for i in range(10):
    x = float(input("Enter a number: "))
    s += x

avg = s / 10
print("Average =", avg)
```

الحالة (ب): إذا كان المقصود معدل الأعداد المتسلسلة من 1 إلى 10

```python
s = 0

for i in range(1, 11):
    s += i

avg = s / 10
print("Average =", avg)
```

المثال الثالث: إيجاد أكبر عدد (`Max`) بين مجموعة من الأعداد

السؤال: `Ex 3: W.P. to find the largest number between a group of numbers`

طريقة التفكير بالحل (مهمة جداً):

1. السؤال قال "مجموعة من الأعداد" ولم يحدد عددها، لذا نسأل المستخدم أولاً: كم عدداً لديك؟ ونخزن الإجابة في `n` .
2. نعمل حلقة `for` تدور `n` من المرات: `for i in range(n):` (تذكر أن `i` ستبدأ من `0` في أول دورة، ثم `1` ، ثم `2` ...).
3. داخل الحلقة يقرأ البرنامج الرقم المدخل `x` .
4. **كيف نعرف الرقم الأكبر؟**
   - في **الدورة الأولى فقط** (`if i == 0:`): لا يوجد لدينا رقم سابق نقارن به! لذلك نعتبر أول رقم يدخله المستخدم هو الأكبر مبدئياً ونخزنه في `max_num = x` .
   - في **بقية الدورات** (`else:`): نقارن كل رقم جديد `x` بالرقم المخزن في `max_num` عبر الشرط `if x > max_num:` . فإذا كان الرقم الجديد أكبر، نحدّث قيمة `max_num` لتصبح مساوية لـ `x` .

PYTHON

```python
n = int(input("How many numbers?: "))

for i in range(n):
    x = int(input("Give x: "))
    if i == 0:
        max_num = x
    else:
        if x > max_num:
            max_num = x

print("The largest number is:", max_num)
```

لماذا استخدمنا `if i == 0:` ولم نضع `max_num = 0` قبل الحلقة؟ لو وضعنا `max_num = 0` قبل الحلقة، ثم أدخل المستخدم أعداداً كلها **سالبة** (مثلاً: `-5` و `-12` و `-3`)، سيقارن البرنامج الأعداد السالبة بالصفر فيجد أن الصفر أكبر منها جميعاً، فيطبع أن أكبر رقم هو `0` (وهو رقم لم يدخله المستخدم أصلاً!). أما طريقة `if i == 0: max_num = x` فهي صحيحة دائماً سواء كانت الأعداد موجبة أو سالبة لأنها تبدأ بأول رقم حقيقي من أرقام المستخدم.

المثال الرابع: إيجاد الأعداد السالبة بين مجموعة من الأعداد

السؤال: `Ex 4: W.P. to find the negative numbers in a group of numbers?`

طريقة التفكير بالحل: العدد السالب في الرياضيات هو كل عدد **أصغر من الصفر** (`x < 0`).

1. نسأل المستخدم عن عدد الأرقام `n` .
2. ننشئ حلقة `for` تدور `n` من المرات، وفي كل دورة ندخل رقماً `x` .
3. نضع شرطاً داخل الحلقة `if x < 0:` لفحص الرقم؛ فإذا كان أصغر من الصفر نطبعه (أو نزيد عدّاد الأعداد السالبة).

الكود الأساسي (لطباعة كل عدد سالب يتم إدخاله):

```python
n = int(input("How many numbers?: "))

for i in range(n):
    x = int(input("Give x: "))
    if x < 0:
        print(x, "is a negative number")
```

تطوير للمثال (حساب عدد الأعداد السالبة ومجموعها في نفس الوقت):

كثيراً ما يطلب الأستاذ في الامتحان: *"أوجد كم عدداً سالباً تم إدخاله (`Count`) أو أوجد مجموع الأعداد السالبة فقط (`Sum`)"*. الحل بسيط جداً:

```python
n = int(input("How many numbers?: "))
count_neg = 0    # عدّاد لحساب كم رقماً سالباً دخل
sum_neg = 0      # وعاء لجمع الأعداد السالبة فقط

for i in range(n):
    x = int(input("Give x: "))
    if x < 0:
        count_neg += 1    # نزيد العداد بمقدار 1 كلما وجدنا رقماً سالباً
        sum_neg += x      # نضيف الرقم السالب إلى مجموع السوالب

print("Count of negative numbers =", count_neg)
print("Sum of negative numbers =", sum_neg)
```

أمثلة إضافية مهمة جداً (تغطي بقية أفكار الامتحانات)

المثال الخامس (إضافي): إيجاد أصغر عدد (`Min`) بين مجموعة أعداد

السؤال: `Ex 5: W.P. to find the smallest number between a group of numbers`

الفكرة: نفس كود أكبر عدد تماماً، مع تغيير إشارة المقارنة من أكبر `>` إلى أصغر `<`!

PYTHON

```python
n = int(input("How many numbers?: "))

for i in range(n):
    x = int(input("Give x: "))
    if i == 0:
        min_num = x
    else:
        if x < min_num:
            min_num = x

print("The smallest number is:", min_num)
```

المثال السادس (إضافي): حساب مضروب العدد (`Factorial`)

السؤال: `Ex 6: W.P. to find the factorial of a number n (n!)`

الفكرة: مضروب العدد (مثل $5! = 1 \times 2 \times 3 \times 4 \times 5 = 120$) هو عملية **ضرب تراكمي** وليس جمعاً تراكمياً:

- في الجمع التراكمي نبدأ بـ `s = 0` (لأن الصفر لا يؤثر في الجمع).
- في الضرب التراكمي **يجب أن نبدأ بـ** `f = 1` (لأننا لو بدأنا بصفر وضربنا فيه لأصبح الناتج صفراً دائماً!).
- نستخدم `range(1, n + 1)` لكي يدخل الرقم `n` ضمن عملية الضرب.

PYTHON

```python
n = int(input("Enter a number to find its factorial: "))
f = 1

for i in range(1, n + 1):
    f *= i    # تكافئ: f = f * i

print("Factorial =", f)
```

المثال السابع (إضافي): جمع الأعداد الزوجية لوحدها والفردية لوحدها من 1 إلى 20

السؤال:

`Ex 7: W.P. to find the sum of even numbers and the sum of odd numbers from 1 to 20`

PYTHON

```python
sum_even = 0
sum_odd = 0

for i in range(1, 21):
    if i % 2 == 0:
        sum_even += i
    else:
        sum_odd += i

print("Sum of Even numbers =", sum_even)
print("Sum of Odd numbers =", sum_odd)
```

📌 ملخص سريع لقوالب الحل في الامتحان (Cheat Sheet)

إذا طُلب منك في السؤال...	القاعدة البرمجية التي تستخدمها فوراً
إدخال عدد من المستخدم	`x = int(input("Enter x: "))`
مجموع تراكمي (`Sum`)	ضع `s = 0` قبل الحلقة، وداخل الحلقة اكتب `s += x`
ضرب تراكمي / مضروب (`Factorial`)	ضع `f = 1` قبل الحلقة، وداخل الحلقة اكتب `f *= i`
حساب عدد مرات تحقق شرط (`Count`)	ضع `c = 0` قبل الحلقة، وداخل الشرط اكتب `c += 1`
حساب المعدل (`Average`)	احسب المجموع `s` أولاً، وبعد انتهاء الحلقة اكتب `avg = s / n`
فحص هل العدد زوجي (`Even`)	`if x % 2 == 0:`
فحص هل العدد فردي (`Odd`)	`if x % 2 ≠ 0:`
فحص هل العدد موجب / سالب	موجب: `if x > 0:` ، سالب: `if x < 0:`
إيجاد أكبر أو أصغر قيمة (`Max / Min`)	في أول دورة `if i == 0:` خزن أول رقم، وفي `else:` قارن وحدّث القيمة

رابعاً: حلقات التكرار المشروطة `while` (`while Loop`)

تُستخدم حلقة `while` لتكرار مجموعة من الأوامر **طالما أن هناك شرطاً معيناً لا يزال صحيحاً** (`True`)، وتتوقف فور أن يصبح هذا الشرط خاطئاً (`False`).

مقارنة جوهرية: متى نستخدم `for` ومتى نستخدم `while` ؟

هذا السؤال يطرحه كل طالب برمجة في الامتحانات:

وجه المقارنة	حلقة `for`	حلقة `while`
طبيعة الاستخدام	تُستخدم عندما يكون **عدد مرات التكرار معلوماً مسبقاً** (مثلاً: 10 مرات، 50 طالباً، من 1 إلى 100).	تُستخدم عندما يكون **عدد مرات التكرار غير معلوم مسبقاً**، ولكن لدينا **شرط توقف أو استمرار** ننتظر حدوثه.
أمثلة واقعية	"اطبع الأرقام من 1 إلى 10" أو "أدخل درجات 5 طلاب".	"استمر في قراءة الأرقام حتى يُدخل المستخدم الرقم صفر"، أو "استمر في طلب كلمة المرور طالما أنها خاطئة".

القاعدة العامة لحلقة `while` وأركانها الثلاثة:

```python
while شرط_الاستمرار:
    الأوامر المراد تكرارها
```

> **الأركان الثلاثة الإلزامية في أي حلقة `while` : كل حلقة `while` صحيحة يجب أن تحتوي على 3 عناصر، ونقصان أي عنصر منها يؤدي إلى خلل أو توقف البرنامج:**
>
> 1. **نقطة البداية (Initialization):** إعطاء قيمة ابتدائية للعداد قبل بدء الحلقة (مثل `x = 1`).
> 2. **شرط الاستمرار (Condition):** الفحص الذي تقرره بايثون قبل كل دورة (مثل `while x < 10:`).
> 3. **خطوة التحديث / التغير (Update / Increment):** تعديل قيمة المتغير داخل الحلقة (مثل `x = x + 1`) ليقترب تدريجياً من شرط التوقف.

الكود الأساسي: طباعة الأرقام من 1 إلى 9

PYTHON

```python
x = 1

while x < 10:
    print(x)
    x = x + 1     # يمكن كتابتها اختصاراً: x += 1
```

- **الناتج على الشاشة:** يطبع الأرقام من `1` إلى `9` .

لماذا توقف البرنامج عند 9 ولم يطبع 10؟

1. يبدأ البرنامج بـ `x = 1` .
2. يفحص الشرط: هل `1 < 10` ؟ نعم (صحيح) ← يطبع `1` ، ثم يزيد `x` لتصبح `2` .
3. يستمر التكرار حتى تصبح `x = 9` : يطبع `9` ، ثم تصبح `x = 10` .
4. في الدورة التالية يفحص الشرط: هل `10 < 10` ؟ **لا (خاطئ)** ← تخرج بايثون من الحلقة فوراً ولا تنفذ الطباعة!

> ⚠ **فخ الحلقة اللانهائية (`Infinite Loop`)! إذا نسي الطالب كتابة سطر التحديث `x = x + 1` ، ستظل قيمة `x` دائماً تساوي `1` ، وسيظل الشرط `1 < 10` صحيحاً للأبد، فيستمر البرنامج بطباعة الرقم 1 دون توقف حتى يتعطل الجهاز أو تتدخل لإيقافه قسرياً!**

تنبيهات وتصحيحات برمجية حاسمة للامتحان:

> ⚠ **3 أخطاء شائعة يقع فيها الطلاب في الامتحانات:**
>
> 1. **خطأ `i++` أو `x++` :** في لغة بايثون **لا يوجد إطلاقاً** شيء اسمه `i++` أو `i--` (هذا موجود في C++ و Java فقط). إذا كتبت `i++` في بايثون سيعطيك البرنامج خطأ (`SyntaxError`). الطريقة الوحيدة الصحيحة هي:
>
> `i = i + 1` *أو* `i += 1`
>
> 2. **رمز "لا يساوي":** في بايثون نكتب علامة التعجب أولاً ثم اليساوي: `!=` وليس `=!` .
> 3. **المسافة البادئة (`Indentation`):** جميع الأسطر التابعة للحلقة يجب أن تبدأ بمسافة بادئة (4 فراغات أو ضغطة زر `Tab`). والأسطر التي تخرج عن المحاذاة تعتبر خارج الحلقة تماماً وتتنفذ بعد نهايتها.

المسألة الأولى (من المحاضرة): معدل الأعداد الزوجية حتى الرقم 14

السؤال:

`Ex 1: W.P. to find the average of even numbers where the last number is 14`

طريقة التفكير وتفكيك المسألة:

1. المستخدم يدخل أعداداً متتالية، ولا نعلم كم عدداً سيدخل، لكننا نعلم شرط التوقف: **نتوقف عندما يُدخل المستخدم الرقم** `14` .
2. معنى ذلك أن **شرط الاستمرار** في الحلقة هو: طالما أن الرقم لا يساوي 14 (`while x ≠ 14:`).
3. المطلوب: حساب **معدل الأعداد الزوجية فقط**.
   - قانون المعدل = **مجموع الأعداد الزوجية (** `sum_even` **) ÷ عدد الأعداد الزوجية (** `count_even` **)**.
   - لذلك نحتاج إلى حصّالة للمجموع تبدأ من الصفر: `sum_even = 0` .
   - ونحتاج إلى عدّاد لحساب كم عدداً زوجياً تم إدخاله: `count_even = 0` (انتبه: العداد يبدأ من `0` وليس `1`).
4. كيف نفحص هل الرقم زوجي؟ بشرط باقي القسمة: `if x % 2 == 0:` .
5. داخل الحلقة، بعد معالجة الرقم الحالي، نطلب من المستخدم إدخال الرقم التالي `x = int(input( ... ))` .

الحل المعتمد أكاديمياً (اعتبار 14 هي رقم التوقف / Sentinel Value):

PYTHON

```python
# الخطوة 1: قراءة أول رقم قبل بدء الحلقة
x = int(input("Enter x: "))

sum_even = 0    # حصالة جمع الأعداد الزوجية
count_even = 0  # عداد لحساب كم رقم زوجي دخل

# الخطوة 2: الاستمرار طالما أن الرقم ليس 14
while x != 14:
    if x % 2 == 0:
        sum_even += x
        count_even += 1

    # قراءة الرقم التالي لكي يستمر الفحص
    x = int(input("Enter x: "))

# الخطوة 3: حساب وطباعة المعدل بعد التوقف
# نتحقق أولاً أن المستخدم أدخل أرقاماً زوجية تجنباً للقسمة على صفر
if count_even > 0:
    avg = sum_even / count_even
    print("The average of even numbers is:", avg)
else:
    print("No even numbers were entered.")
```

ماذا لو أراد الأستاذ احتساب الرقم 14 نفسه ضمن الأعداد؟ إذا كان قصد الأستاذ أن الرقم 14 هو آخر رقم يدخل في الحساب، فالحل بسيط جداً: بما أن 14 عدد زوجي، نضيفه يدوياً بعد خروج الحلقة:

PYTHON

```python
sum_even += 14
count_even += 1
avg = sum_even / count_even
print("The average is:", avg)
```

المسألة الثانية (من المحاضرة): حاصل ضرب الأعداد حتى يصبح الرقم أكبر من سابقه بمقدار 2

السؤال:

```
Ex 2: W.P. to find the multiplication of numbers where the last number is
                                                    greater than its previous by two
```

طريقة التفكير وتفكيك المسألة (فكرة المقارنة بين الحالي والسابق):

1. المسألة تطلب إيجاد **حاصل ضرب** أعداد يدخلها المستخدم:
   - الضرب يحتاج إلى متغير يبدأ بـ `1` وليس صفراً: `m = 1` .
2. شرط التوقف في السؤال: "عندما يكون الرقم الأخير أكبر من الرقم السابق له بمقدار 2".
   - رياضياً: إذا كان الرقم الحالي اسمه `y` والرقم السابق اسمه `x` ، فإن شرط التوقف هو:

```
x - y == 2
```

   - وبما أن حلقة `while` تحتاج إلى **شرط الاستمرار**، فنحن نستمر طالما أن الفرق لا يساوي 2:

```
x - y != 2
```

3. لكي نبدأ بمقارنة رقم برقم سابق له، **يجب أن نقرأ أول رقمين من المستخدم قبل بدء الحلقة**:
   - نقرأ الرقم الأول ونعتبره السابق: `x` .
   - نقرأ الرقم الثاني ونعتبره الحالي: `y` .
   - نضيفهما إلى حاصل الضرب.
4. داخل الحلقة:
   - نحدّث المتغيرات: الرقم الذي كان "حالياً" يصبح هو "السابق" للدورة التالية (`x = y`).
   - نطلب من المستخدم إدخال رقم حالي جديد (`x = int( ... )`).
   - نضربه في المجموع التراكمي.

الكود البرمجي الكامل:

```python
M = 1

# قراءة أول رقمين للبدء بالمقارنة
x = int(input("Enter first number: "))
m *= x

y = int(input("Enter second number: "))
m *= y

# نستمر طالما أن الرقم الحالي ليس أكبر من السابق بمقدار 2
while x - y != 2:
    x = y                          # يصبح الحالي هو السابق للدورة القادمة
    x = int(input("Enter next number: "))  # إدخال رقم جديد
    m *= x                         # ضرب الرقم الجديد

print("Condition met! The multiplication is:", product)
```

تجربة الكود وتتبعه بالأرقام:

- أدخل المستخدم أولاً: `y = 5` ← الضرب: `1 * 5 = 5`
- أدخل المستخدم ثانياً: `x = 6` ← الضرب: `5 * 6 = 30`
- فحص الشرط: هل `6 - 5 == 2` ؟ لا (الفرق 1) ← يستمر في الحلقة.
- يصبح `y = 6` ، ويدخل المستخدم رقماً جديداً: `x = 8` .
- الضرب: `30 * 8 = 240` .
- فحص الشرط: هل `8 - 6 == 2` ؟ **نعم! (الفرق 2)** ← يتوقف البرنامج فوراً ويطبع: `Condition met! The multiplication is: 240` .

أمثلة إضافية مهمة جداً على `while` في الامتحانات

مثال إضافي 1: جمع الأرقام حتى يُدخل المستخدم صفر (المثال الكلاسيكي الأشهر)

السؤال:

```
W.P. to find the sum of positive numbers entered by the user until the user enters 0.
```

```python
total = 0
x = int(input("Enter a number (0 to stop): "))

while x != 0:
    total += x
    x = int(input("Enter a number (0 to stop): "))

print("Total sum =", total)
```

مثال إضافي 2: عدّ خانات العدد أو استخراج الأرقام (Digits Counting)

السؤال:

`W.P. to count the number of digits in an integer (e.g., 5834 has 4 digits).`

الفكرة الرياضية: عند قسمة أي عدد صحيح على `10` قسمة صحيحة (`// 10`)، فإننا نحذف خانة الآحاد منه (مثلاً: `583 // 10 = 58`). فنستمر بحذف الخانات داخل `while` ونزيد العداد في كل مرة حتى يصبح العدد صفراً!

```python
num = int(input("Enter a number: "))
count = 0

# نحفظ نسخة من الرقم للطباعة
temp = num

while temp > 0:
    temp = temp // 10    # حذف خانة الآحاد
    count += 1           # زيادة عداد الخانات

print("The number of digits in", num, "is:", count)
```

مثال إضافي 3: التحقق من صحة المدخلات (Input Validation)

السؤال: اطلب من المستخدم إدخال رقم موجب حصراً، وإذا أدخل رقماً سالباً أعد الطلب منه حتى يلتزم.

```python
x = int(input("Enter a positive number: "))

# طالما أن الرقم سالب أو صفر، كرر رسالة الخطأ
while x <= 0:
    print("Invalid! The number must be positive.")
    x = int(input("Try again, enter a positive number: "))

print("Thank you! You entered:", x)
```

أوامر التحكم الإضافية داخل الحلقات: `break` و `continue`

أحياناً نحتاج للتحكم بمسار الحلقة من الداخل:

1. **الأمر `break` (الخروج القسري):** يقطع الحلقة فوراً ويخرج منها حتى لو كان شرط الـ `while` ما يزال صحيحاً.
2. **الأمر `continue` (تجاوز الدورة الحالية):** يتجاهل ما تبقى من أسطر في الدورة الحالية ويقفز فوراً لبدء الدورة التالية.

```python
# مثال على break: التوقف عند الرقم 5
i = 1
while i <= 10:
    if i == 5:
        break    # يتوقف تماماً عندما يصل 5
    print(i)
    i += 1
# الناتج: يطبع 1, 2, 3, 4 فقط
```

📌 ملخص سريع للتعامل مع `while` في ورقة الامتحان:

1. **هل عدد الدورات محدد في السؤال؟**
   - نعم ← استخدم `for i in range(...)`.
   - لا (يوجد شرط توقف عند رقم معين أو فرق معين) ← استخدم `while`.
2. **خطوات الحل الثابتة مع `while`:**
   - اقرأ أول قيمة من المستخدم قبل السطر الخاص بـ `while`.

- اكتب عكس شرط التوقف في سطر `while` (لأنه شرط استمرار).
- قم بالعمليات الحسابية (`sum` , `product` , `count`).
- **لا تنسَ إطلاقاً** قراءة القيمة التالية أو زيادة العداد في آخر سطر داخل الحلقة.